TECHNICAL INTERVIEW PREP · 2 DE 10

# Technical Interview Questions - Part 2 - Typescript

Giuliana Olmos · 24 de octubre de 2024 · https://dev.to/giulianaolmos/technical-interview-questions-part-2-typescript-1njn

## Introduction

Hello, hello!! :D

Hope you're all doing well!

How we're really feeling:

I'm back with the second part of this series. 🤓

In this chapter, we'll dive into the ✨Typescript✨ questions I've faced during interviews.

I'll keep the intro short, so let's jump right in!

## Questions

1. What are generics in typescript? What is `<T>`?
2. What are the differences between interfaces and types?
3. What are the differences between `any`, `null`, `unknown`, and `never`?

---

### Question 1: What are generics in typescript? What is `<T>`?

*The short answer is...*

> Generics in TypeScript allow us to create reusable functions, classes, and interfaces that can work with a variety of types, without having to specify a particular one. This helps to avoid using `any` as a catch-all type.

> The syntax `<T>` is used to declare a generic type, but you could also use `<V>`, `<M>`, or any other placeholder.

*How does it work?*

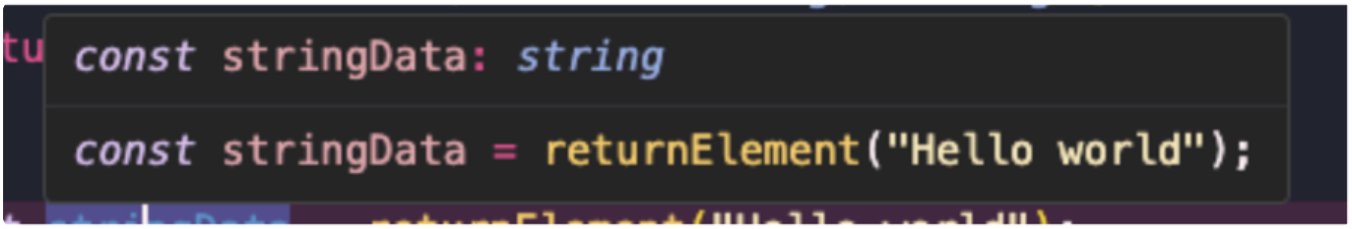
Let's break it down with an example.

Suppose I have a function that accepts a parameter and returns an element of the same type. If I write that function with a specific type, it would look like this:

```
function returnElement(element: string): string {
 return element;
}


const stringData = returnElement("Hello world");
```

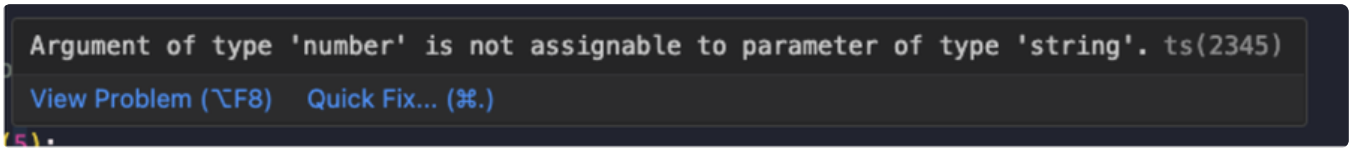
I know the type of `stringData` will be “string” because I declared it.




But what happens if I want to return a different type?

```
const numberData = returnElement(5);
```

I will receive an error message because the type differs from what was declared.




The solution could be to create a new function to return a number type.

```
function returnNumber(element: number): number {
 return element;
}
```

That approach would work, but it could lead to duplicated code.

A common mistake to avoid this is using `any` instead of a declared type, but that defeats the purpose of type safety.

```
function returnElement2(element: any): any {
 return element;
}
```

However, using `any` causes us to lose the type safety and error detection feature that Typescript has. Also, if you start using `any` whenever you need to avoid duplicate code, your code will lose maintainability.

This is precisely when it’s beneficial to use generics.

```
function returnGenericElement<T>(element: T): T {
 return element;
}
```

The function will receive an element of a specific type; that type will replace the generic and remain so throughout the runtime.

This approach enables us to eliminate duplicated code while preserving type safety.

```
const stringData2 = returnGenericElement("Hello world");


const numberData2 = returnGenericElement(5);
```

*But what if I need a specific function that comes from an array?*

We could declare the generic as an array and write it like this:

```
function returnLength<T>(element: T[]): number {
 return element.length;
}
```

Then,

```
const stringLength = returnLength(["Hello", "world"]);
```

The declared types will be replaced by the type provided as a parameter.

```
function returnLength<string>(element: string[]): number
```

*We can also use generics in classes.*

```
class Addition<U> {
 add: (x: U, y: U) => U;
}
```

I have three points to make about this code:

1. `add` is an anonymous arrow function (which I discussed in the first chapter).
2. The generic can be named `<U>`, `<T>`, or even `<Banana>`, if you prefer.

3. Since we haven't specified the type yet, we can't implement operations inside the classes. Therefore, we need to instantiate the class by declaring the type of the generic and then implement the function.

Here's how it looks:

```
const operation = new Addition<number>();


operation.add = (x, y) => x + y; => We implement the function here


console.log(operation.add(5, 6)); // 11
```

And, one last thing to add before ending this question. Remember that generics are a feature of Typescript. That means the generics will be erased when we compile it into Javascript.

From

```
function returnGenericElement<T>(element: T): T {
 return element;
}
```

to

```
function returnGenericElement(element) {
 return element;
}
```

---

## Question 2: What are the differences between interfaces and types?

*The short answer is:*

> 1. Declaration merging works with interfaces but not with types.
> 2. You cannot use `implements` in a class with union types.
> 3. You cannot use `extends` with an interface using union types.

Regarding the first point, *what do I mean by declaration merging?*

Let me show you: I've defined the same interface twice while using it in a class. The class will then incorporate the properties declared in both definitions.

```ts
interface CatInterface {
 name: string;
 age: number;
}


interface CatInterface {
 color: string;
}


const cat: CatInterface = {
 name: "Tom",
 age: 5,
 color: "Black",
};
```

This does not occur with types. If we attempt to define a `type` more than once, TypeScript will throw an error.

```ts
type dog = {
 name: string;
 age: number;
};


type dog = { // Duplicate identifier 'dog'.ts(2300)
 color: string;
};


const dog1: dog = {
 name: "Tom",
 age: 5,
 color: "Black", //Object literal may only specify known properties, and 'color' does not exist in type 'dog'.ts(2353)
};
```



```
Object literal may only specify known properties, and 'color' does not exist in type 'dog'. ts(2353)
(property) color: string
```

Regarding the following points, let's differentiate between `union` and `intersection` types:

`Union types` allow us to specify that a value can be one of several types. This is useful when a variable can hold multiple types.

`Intersection types` allow us to combine types into one. It is defined using the `&` operator.

```
type cat = {
 name: string;
 age: number;
};


type dog = {
 name: string;
 age: number;
 breed: string;
};
```

**Union type:**

```
type animal = cat | dog;
```

**Intersection type:**

```
type intersectionAnimal = cat & dog;
```

If we attempt to use the `implements` keyword with a `union type`, such as `Animal`, TypeScript will throw an error. This is because `implements` expects a single interface or type, rather than a union type.

```
class pet implements animal{
   name: string;
   age: number;
   breed: string;
   constructor(name: string, age: number, breed: string){
       this.name = name;
       this.age = age;
       this.breed = breed;
   }
}
```

```
A class can only implement an object type or intersection of object types with statically known members. ts(2422)
type animal = cat | dog
```

***Typescript allows us to use “implements” with:***

a. Intersection types

```
class pet2 implements intersectionAnimal {
 name: string;
 age: number;
 color: string;
 breed: string;
 constructor(name: string, age: number, color: string, breed: string) {
   this.name = name;
   this.age = age;
   this.color = color;
   this.breed = breed;
 }
}
```

b. Interfaces

```
interface CatInterface {
 name: string;
 age: number;
 color: string;
}
```

```
class pet3 implements CatInterface {
 name: string;
 age: number;
 color: string;
 constructor(name: string, age: number, color: string) {
   this.name = name;
   this.age = age;
   this.color = color;
 }
}
```

c. Single Type.

```
class pet4 implements cat {
 name: string;
 age: number;
 color: string;
 constructor(name: string, age: number, color: string) {
   this.name = name;
   this.age = age;
```

```
    this.color = color;
  }
}
```

The same issue occurs when we try to use `extends` with a union type. TypeScript will throw an error because an `interface` cannot extend a `union type.` Here’s an example

```
interface petUnionType extends animal {
 name: string;
 age: number;
 breed: string;
}
```

You cannot extend a union type because it represents multiple possible types, and it's unclear which type's properties should be inherited.

```
An interface can only extend an object type or intersection of object types with statically known members. ts(2312)
type animal = cat | dog
```

BUT you can extend a `type` or an `interface`.

```
interface petIntersectionType extends intersectionAnimal {
 name: string;
 age: number;
 color: string;
 breed: string;
}


interface petCatInterface extends CatInterface {
 name: string;
 age: number;
 color: string;
}
```

Also, you can extend a single type.

```
interface petCatType extends cat {
   name: string;
   age: number;
   color: string;
   }
```

## Question 3: What are the differences between `any`, `null`, `unknown`, and `never`?

*Short answer:*

> `Any` => It's a top-type variable (also called universal type or universal supertype). When we use `any` in a variable, the variable could hold any type. It's typically used when the specific type of a variable is unknown or expected to change. However, using any is not considered a best practice; it's recommended to use generics instead.

```
let anyVariable: any;
```

While `any` allows for operations like calling methods, the TypeScript compiler won't catch errors at this stage. For instance:

```
anyVariable.trim();
anyVariable.length;
```

You can assign any value to an `any` variable:

```
anyVariable = 5;
anyVariable = "Hello";
```

Furthermore, you can assign an `any` variable to another variable with a defined type:

```
let booleanVariable: boolean = anyVariable;
let numberVariable: number = anyVariable;
```

---

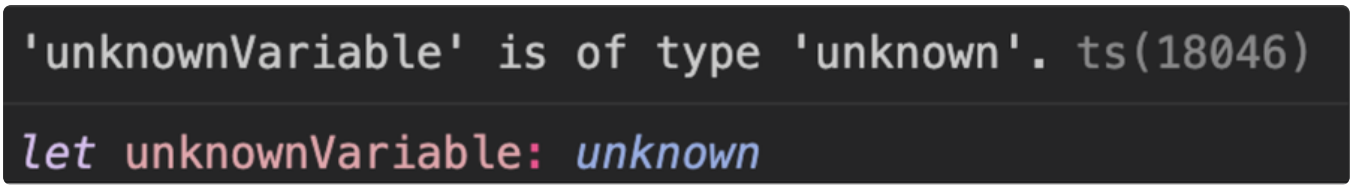
> `Unknown` => This type, like `any`, could hold any value and is also considered the top type. We use it when we don't know the variable type, but it will be assigned later and remain the same during the runtime. `Unknow` is a less permissive type than `any`.

```
let unknownVariable: unknown;
```

Directly calling methods on unknown will result in a compile-time error:

```
unknownVariable.trim();
```

```
unknownVariable.length;
```




Before using it, we should perform checks like:

```
if (typeof unknownVariable === "string") {
 unknownVariable.trim();
}
```

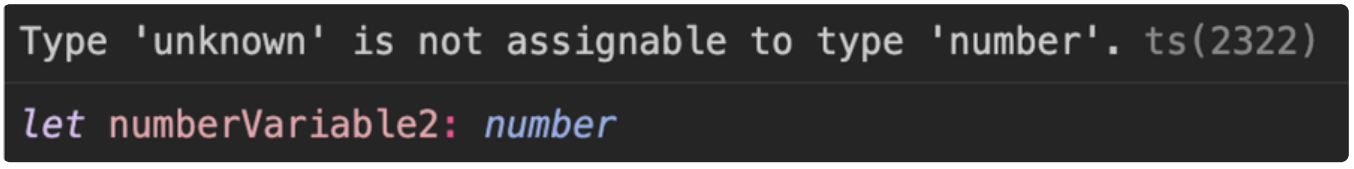
Like `any,` we could assign any type to the variable.

```
unknownVariable = 5;
unknownVariable = "Hello";
```

However, we cannot assign the `unknown` type to another type, but `any` or `unknown`.

```
let booleanVariable2: boolean = unknownVariable;
let numberVariable2: number = unknownVariable;
```

This will show us an error




---

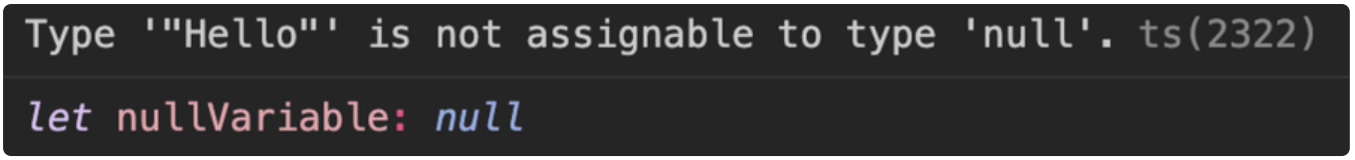
> `Null` => The variable can hold either type. It means that the variable does not have a value.

```
let nullVariable: null;

nullVariable = null;
```

Attempting to assign any other type to a null variable will result in an error:

```
nullVariable = "Hello";
```




---

> `Never` => We use this `type` to specify that a function doesn't have a return value.

```
function throwError(message: string): never {
 throw new Error(message);
}


function infiniteLoop(): never {
 while (true) {
   console.log("Hello");
 }
}
```

---

## The end...

We finish with Typescript,

For today (?

I hope this was helpful to someone.

If you have any technical interview questions you'd like me to explain, feel free to let me know in the comments. 🫶

Have a great week 💖